computers

MDPI

Review

AI-Powered Software Development: A Systematic Review of Recommender Systems for Programmers

Efthimia Mavridou [1], Eleni Vrochidou [1], Theofanis Kalampokas [1], Venetis Kanakaris [2] and George A. Papakostas [1,*]

[1] MLV Research Group, Department of Informatics, Democritus University of Thrace, 65404 Kavala, Greece; emavridou@cs.duth.gr (E.M.); evrochid@cs.duth.gr (E.V.); tkalampo@cs.duth.gr (T.K.)
[2] Department of Economics, Democritus University of Thrace, 69100 Komotini, Greece; vkanakar@econ.duth.gr
* Correspondence: gpapak@cs.duth.gr; Tel.: +30-2510462321

Abstract: Software engineering is a field that demands extensive knowledge and involves numerous challenges in managing information. The information landscapes in software engineering encompass source code and its revision history, a set of explicit instructions for writing, commenting on and running the codes, a set of procedures and routines, and the development environment. For software engineers who develop code, writing code documentation is also extremely important. Due to the technical complexity, vast scale, and dynamic nature of software engineering, there is a need for a specialized category of tools to assist developers, known as recommendation systems in software engineering (RSSE). RSSEs are specialized software applications designed to assist developers by providing valuable resources, code snippets, solutions to problems, and other useful information and suggestions tailored to their specific tasks. Through the analysis of data and user interactions, RSSEs aim to enhance productivity and decision-making for developers. To this end, this work presents an analysis of the literature on recommender systems for programmers, highlighting the distinct attributes of RSSEs. Moreover, it summarizes all related challenges regarding developing, assessing, and utilizing RSSEs, and offers a broad perspective on the present state of research and advancements in recommendation systems for the highly technical field of software engineering.

Keywords: recommender systems; AI-driven recommenders; software engineering; programming; code suggestions; intelligent software tools; code completion

check for updates

Academic Editor: Paolo Bellavista

Received: 18 February 2025
Revised: 13 March 2025
Accepted: 20 March 2025
Published: 24 March 2025

Citation: Mavridou, E.; Vrochidou, E.; Kalampokas, T.; Kanakaris, V.; Papakostas, G.A. AI-Powered Software Development: A Systematic Review of Recommender Systems for Programmers. *Computers* **2025**, *14*, 119. https://doi.org/10.3390/computers14040119

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Recommender systems assist users in tackling information overload by providing relevant suggestions. They have been successfully applied in many domains like e-commerce and education. In software engineering, with the rapid evolution of frameworks, programming languages and libraries and given the complexity of the field, there is a great demand for systems that assist programmers in working more efficiently and effectively. Note that it has been verified [1] that software developers spend about 19% of their time looking for code examples to use their code.

Recommender systems for software engineering have been proposed to assist users with the development process [2]. A recommendation system for software engineering (RSSE) is as a software application that provides information items estimated to be valuable for a software engineering task in a given context [2]. Recommenders can assist developers in various tasks during different phases of the development lifecycle, i.e., requirement analysis, designing, programming, and testing. Most of the existing RSSEs focus on

supporting developers in the implementation phases. Recommender systems can assist developers with suggesting relevant code snippets for developing new code, refactoring existing code, fixing bugs, etc. [3].

Most of the research on RSSEs involves mining large code repositories in order to generate code suggestions. The rapid growth of software repositories and the open-source paradigm have made available a vast amount of code that RSSEs leverage for learning coding patterns and providing recommendations. RSSEs differ from traditional recommender systems in that they are more task-centric rather than user-centric. However, personalization is widely used in recommender systems in other domains with great success, such as e-learning [4] and e-commerce [5]. For instance, adaptive learning platforms like Khan Academy analyze learners' activities, recommending specific lessons, practice exercises, and resources tailored to their needs, offering a continuous and personalized learning experience. Large commerce web sites, like Amazon, also employ recommender systems towards helping their customers find the appropriate products to purchase, by learning from them and creating recommendations based on their needs.

To this end, this survey aims to review the current state of recommender systems for programmers by conducting an analysis of the literature on the topic. This work aims to present comparative results, along with crucial related aspects, such as the most used tasks to assist developers, the user input they require, the possible output representations, the methods used towards generating recommendations, the capability to consider user-specific information for providing more personalized recommendations, the faced challenges and how they are surpassed, and how trends change in coverage of certain issues. The main objective of this work is, through the conducted literature analysis, to bring the theoretical part of related works in line with the practical part and justify its importance. Based on the analysis, we foresee identifying the main challenges and future research directions for RSSEs for programming.

The rest of the paper is structured as follows. Section 2 presents related works and highlights the contributions of the present survey. Section 3 summarizes the research methodology. Recommender systems for programmers are presented in Section 4, Section 5 discusses the results and Section 6 concludes the paper and highlights future directions.

2. Related Work

Related works refer to survey articles on recommenders for programming that can be found in the academic literature. Research efforts on RSSEs regarding design, implementation, and evaluation were presented in the book of Robillar et al. [2]. However, their work does not contain recent research efforts since it was published 10 years ago, in 2014. A survey on recommenders for software engineering published the same year, focusing on how RSSEs can assist users in each development lifecycle, was published by Pakdeetrakulwong et al. [6]. It reviewed 23 publications between 2004 and 2011. Thus, it does not contain any recent information.

A review that examined the functionality offered by existing RSSEs evaluating 46 published between 2003 and 2013 was presented in [5]. The latter review is the closest to our survey. However, the reviewed research efforts were over 10 years ago. Therefore, a recent systematic review focusing on recommenders for programming is considered necessary. There are some recent surveys that are related to the subject; however, they focus on different aspects and not specifically on recommender systems for programming. For example, the authors in [7] examined the use of AI in Software Engineering (SE) phases from 2013 to 2023. Similarly, a comprehensive literature review on deep learning (DL) for code intelligence was presented in [8].

Compared to related works, our survey focuses solely on recommender systems for programming. To this end, the aim of this survey is to present an overview of the recent research efforts on recommender systems for programming, highlighting their main characteristics, challenges and potential future research directions, covering the most recent years up to 2024, where a gap in the literature has been identified.

3. Research Methodology

Research questions (RQs) are considered essential for ensuring that the survey article is well-organized, focused, and informative. RQs can guide the entire research process and help in achieving the article's objectives effectively, towards identifying key themes, addressing research gaps and informing the audience about the purpose and significance of the study. The RQs that guided the present research are the following:

- RQ1: What do RSSEs assist users with?
- RQ2: What user inputs do RSSEs require to make recommendations?
- RQ3: What output do RSSEs present to the user?
- RQ4: How are the recommendations generated?
- RQ5: Do they consider user context and personal information?
- RQ6: What topics are problematic and how are they covered?
- RQ7: How do trends change in the coverage of certain issues?

This review was performed in accordance with the PRISMA (Preferred Reporting Items for Systematic Reviews and Meta-Analyses) guidelines. More specifically, the PRISMA-ScR methodology was applied to develop the survey. It is an extension of the well-known PRISMA for reviews. The bibliographic database Scopus was used for research since it includes papers from important libraries such as Springer, Elsevier, IEEE, etc. The query posed to Scopus on 30 January 2025 was as follows:

Search within article title, abstract, keywords: TITLE-ABS-KEY ("code recommendation" OR "code suggestion" OR "programming assistant" OR "source code recommendation" OR "code refactoring" OR "code completion") AND ("software engineering" OR "IDEs" OR "development environment" OR "programming" OR "software development") AND ("recommender" OR "personalized" OR "personalization" OR "personalisation" OR "user modelling" OR "context-aware") AND PUBYEAR > 2014

The Scopus query returned 745 records. In the first screening, we excluded those irrelevant to the topic. Moreover, papers not written in English or not accessible were also removed. After that, 312 records remained. During the second screening, papers that did not comply with research questions were removed. For example, a great number of papers were about automatic program repair and bug detection. However, since those approaches are not recommenders, they were removed from our dataset. Thus, 50 papers remained and were included in our survey. The flow diagram of the followed PRISMA methodology is illustrated in Figure 1.

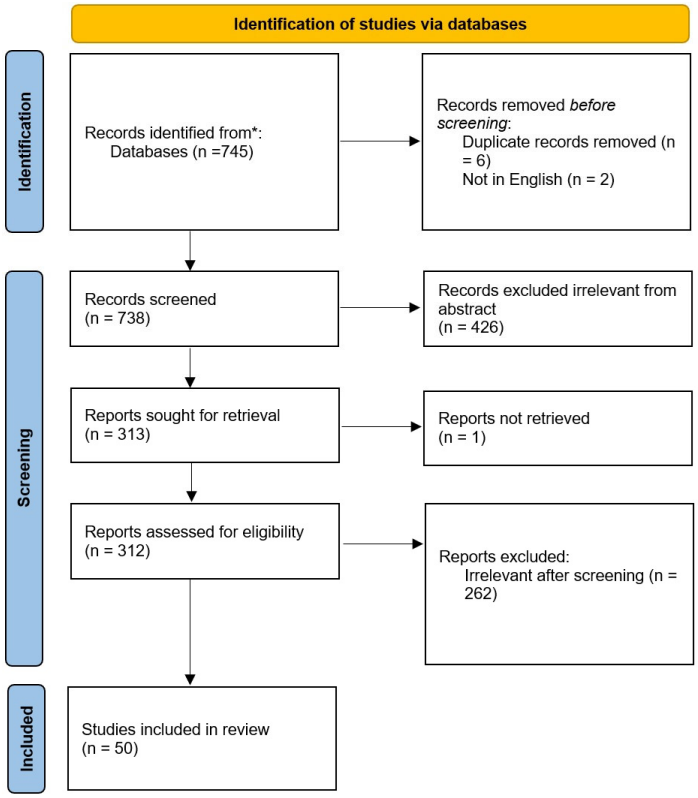

Figure 1. PRISMA flow diagram.

4. Recommender Systems for Programmers

Recommender systems can assist programmers in all phases of the software development lifecycle by providing valuable information items. Most of those systems focus on the implementation phase, aiming to assist developers with programming. Developers tend to use existing code when programming. In this way, they do not have to start from scratch and can develop faster and more efficiently. However, finding the right code to use is not always easy [1]. To this end, a lot of research efforts focused on code recommendation systems. Those systems recommend code snippets to developers mainly based on their code context. It differs from code generation, which refers to automatically creating code from user input. However, recommendation techniques can leverage code generation for suggesting code to developers.

Code completion is a similar recommendation task that involves predicting the next code token(s) based on its contextual information within the code [9]. The main difference with code recommendation is that code is recommended to the user while they type.

Developers spend a great amount of their time fixing bugs and refactoring code already written. Recommender systems that suggest bug fixes and refactoring snippets can assist developers in repairing code faster and more efficiently. Many research efforts have focused on automatic bug repair. However, those systems automatically apply bug repair fixes rather than suggesting to developers the code to use. Similarly, code search is basically an information retrieval system rather than a recommender. To this end, those efforts are not included in our review since they are not recommender systems.

The retrieved papers (after the exclusion of the irrelevant ones) were separated based on the main way they help developers (RQ1). Four main categories were identified, namely code recommendation, code completion, recommending repairs and API recommendation. Recommending repairs category includes works on refactoring and recommending bug fixes. Application Programming Interfaces (APIs) are mechanisms that allow communica-

tion between two programs [9]. API recommendation systems suggest to developers APIs to use in their code. Thus, those systems focus on recommending the right API for a specific task. There are also recommenders that assist developers in using APIs by recommending API arguments or API methods to use and in what order. Most of the retrieved papers for this review are API recommender systems.

Figure 2 shows the categories identified along with their size. As it is depicted in Figure 2, almost half of the retrieved papers belong to the category "API recommendation". The rest of the categories identified are code recommendation, code completion and code repair recommendation, which are about similar size to code recommendation, with the largest number of related papers.

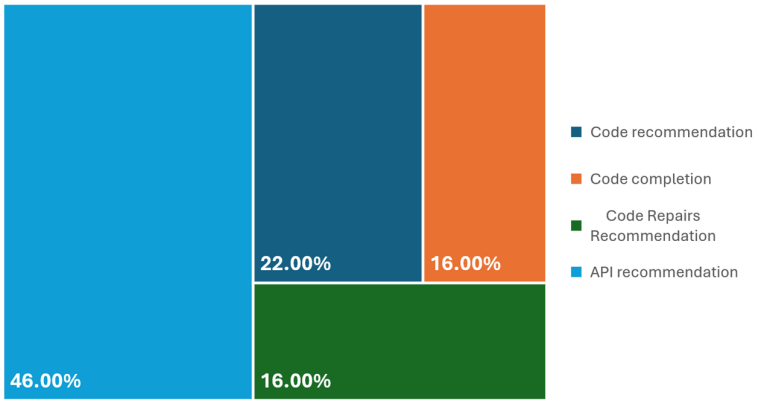

Figure 2. Aim of RSSEs (RQ1).

It is worth noticing that the number of papers per year varies chronologically. As depicted in Figure 3, research on API recommendation can be found almost every year in the 10-year span of our research, indicating it is an active and established area of research. The same stands for code recommendation. However, it seems that less research has been published in this area compared to API recommendations. Code completion publications included in our survey were published after 2022. Moreover, the number of code completion publications per year increases exponentially. The recent advances in LLMs and deep learning architectures have driven research in many areas. Code completion leverages LLMs to suggest code while the user types. Thus, the increased number of published papers in these areas is inevitable.

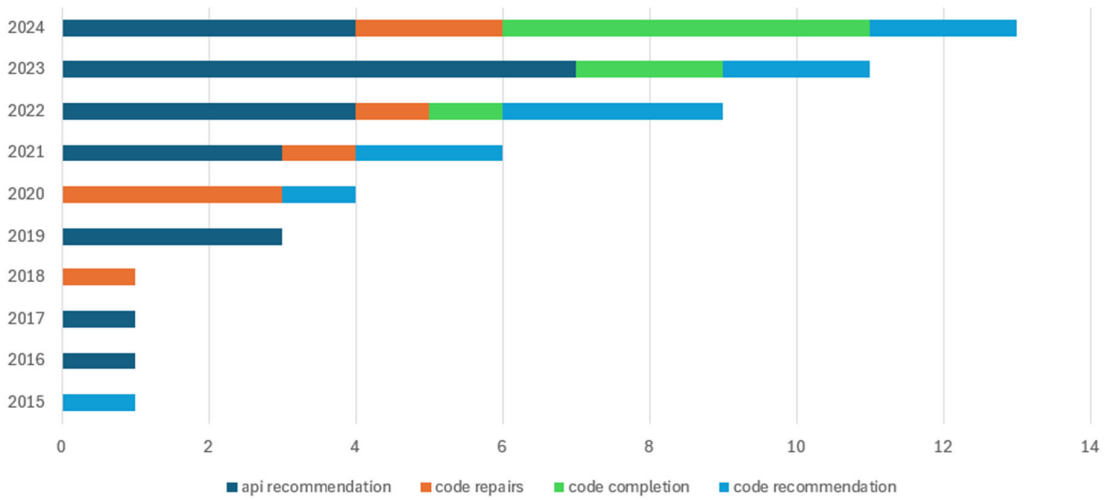

Figure 3. Number of reviewed publications per year.

4.1. Code Recommendation

Code recommendation involves suggesting code to use while programming, which can drastically improve developers' productivity if performed correctly. It basically refers to recommending code based on the source code context. It differs from code search, which refers to the developers searching for code snippets [10].

The current advanced techniques in code recommendation mostly focus on utilizing large code repositories for extracting common code patterns and making recommendations based on them. A code recommendation mechanism based on self-attention neural networks is proposed in [11] that extracts both lexical and syntactical information to capture the deep semantics of code and finally recommend the relevant code to developers. Specifically, both bag-of words and Abstract Syntax Tree (AST) representation of the developer's code and candidate snippets from the codebase were used to train the neural network (NN) and finally perform recommendations based on the source code of the current user. Similarly, a code recommendation engine that recommends relevant snippets based on AST representation was presented in [12]. It uses a novel method, namely De-Skew Locality Sensitive Hashing (De-Skew LSH), that allows for faster and more accurate recommendations even when the length of the query largely differs from the length of code snippets.

A recommender that suggests code clones based on deep learning and information retrieval was discussed in [13]. Code clones are similar code fragments that represent functionalities used repeatedly and thus are often suitable for use in other projects.

Several approaches recommend code snippets based on the user's query code or natural language query. The recommendation method in [14] presented recommends code snippets based on users' language queries. A code corpus is constructed by filtering and cleaning data from code datasets. The user query and the code snippets are mapped into the same vector space through the Sentence Bidirectional Encoder Representations from Transformers (SBERT) model. Similarity, matching is then performed between the vectors and the most similar are recommended to the user.

Similarly, the recommender system in [15] recommends code snippets based on the user's natural language query. The Gated Recurrent Unit (GRU) Network is deployed to embed code snippets and described queries into a vector representation. The code recommendation was performed based on similarity matching with the use of the Joint Embedded Attention Network (JEAN) model. A code recommendation technique that suggests code from private repositories instead of using data from public repositories and Questions and Answers (Q&A) websites was proposed in [16]. It is based on similarity matching with the user's natural language query. It suggests queries based on the initial text query to recommend relevant code snippets.

Some approaches have addressed the problem of recommending the next-to-implement code. The work presented in [17] deals with a code recommender system, namely CA-FACER, that supports opportunistic reuse [18], which suggests code based on the features a developer may want to implement next. CA-FACER is implemented as a plugin in the Intellij IDEA IDE that finds relevant projects and recommends their popular features based on their similarity with the developer's active project. In an earlier work [19], FACER provided relevant code recommendations for opportunistic reuse based on the recent method searched. An approach for predicting the functionality of incomplete programming code was proposed in [20]. It relies on a deep learning model and AST-based representation of the code that captures syntactical and sequential information.

Towards more user-driven and personalized recommendations, PERSONA, a code recommender system that considers the personal coding patterns of developers, was presented in [21]. PERSONA recommends code elements (variable names, class names, methods, and parameters) based on each developer's coding history while also combining

project-specific and common code patterns. A recommendation technique for suggesting method parameters based on the current usage context was proposed in [22]. It is based on similarity matching considering the four lines before the method invocation.

Finally, a recommendation framework for transformer-based code generation models with the aim of providing more secure and high-quality code recommendations was described in [23]. It relies on heuristics-based filtering and quality ranking scoring.

Regarding RQ2, Table 1 summarizes the results for the publications that refer to the code recommendation task. Research efforts in [11–13,20,22] take as input the current user code. On the contrary, research presented in [14–16,23] is based on natural language queries. Thus, they require input from developers to generate recommendations. Developers' coding history and code patterns used throughout the project are considered in [21].

Table 1. Input of recommender system—code recommendation (RQ2).

Ref.	Input
[11–13,20,22]	Current code segment
[14–16,23]	Natural language query
[17]	Methods with API usages in a current active project
[21]	Developers' coding history, project-specific and common code patterns

Figure 4 summarizes the input types required to generate recommendations for the code recommendation category. Almost half of the studies use as input a natural language query (46%). In total, 36% of the publications of this category use the current code segment.

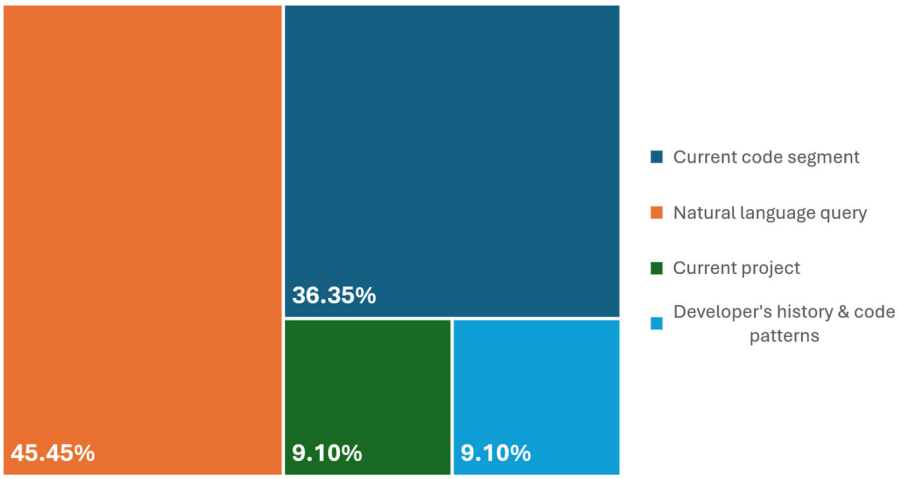

Figure 4. User input for code recommendation (RQ2).

Regarding RQ3, the RSSEs presented in [11–16,23] recommend code snippets to the end user. The RSSE presented in [21] recommends variable names, class names, methods and parameters based on the personal coding patterns of developers. Code clone methods are recommended in [13]. The recommender system presented in [20] provides a list of potential functionalities based on the current user code. Table 2 includes the output of RSSE systems in the code recommendation category.

Table 2. Output of recommender system—code recommendation (RQ3).

Ref.	Output
[11–16,23]	Ranked list of code snippets
[17]	Ranked list of related methods
[20]	A list of potential functionalities
[21]	Ranked list of code elements
[22]	Method parameters

Figure 5 gives an overview of the distribution of outputs. The majority of the code recommendation approaches (63.64%) return a ranked list of code snippets.

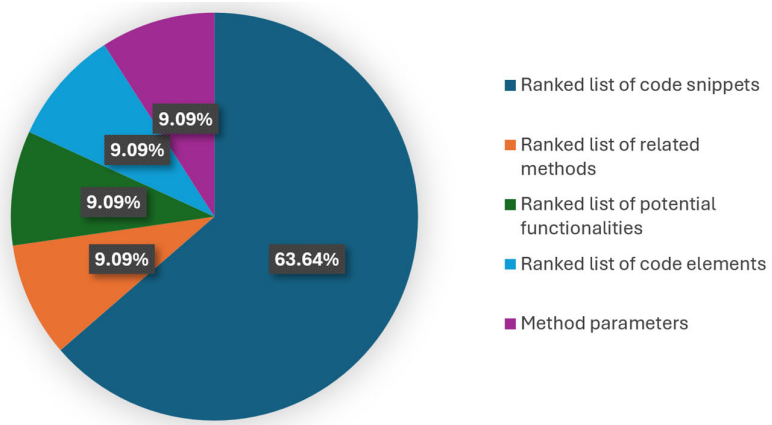

Figure 5. User output for code recommendation (RQ3).

Regarding used methods (RQ3), authors in [12] apply similarity matching between the query code and candidate code snippets. Pretrained LLMs were used in [13]. A self-attention neural network is deployed in [11] and a GRU Network and JEAN model in [15], while in [21], the researchers leverage fuzzy logic. Table 3 summarizes the methods used for generating recommendations based on the selected literature.

Table 3. Methods used for generating the recommendations—code recommendation (RQ3).

Ref.	Method
[11]	Self-attention neural networks
[12]	AST-based feature scoring and De-Skew LSH
[13]	Pretrained GPT-2 and IR technique (TF-IDF)
[14]	SBERT model and similarity matching
[15]	GRU Network and JEAN model
[16]	Similarity matching
[17]	Similarity matching (cosine similarity)
[20]	Deep learning model
[21]	Fuzzy sets
[23]	Heuristics and ranking based on quality criteria applied for the output of transformer-based code generation model
[22]	Similarity matching (cosine similarity and locality sensitive hashing)

Regarding the consideration of user content and personal information, our survey identified that most of the works take as input only the source code of the developer, while a few require input from a natural language query. The only study of this category that considers the developer's coding history providing more personalized recommendations is the one presented in [21].

4.2. Code Completion

Code completion involves recommending the next code tokens that complete the code that is recently typed [9]. A deep learning model based on a combination of attention and gated highway mechanism that suggests the next code token was presented in [24]. Code Large Language Models (LLMs), such as Codex [25] and Code Llama [26], have demonstrated impressive capability in general code completion tasks [27]. They are based on transformer neural networks and have been trained in vast code corpora. Modern IDEs include these models in autocompletion plugins such as GitHub Copilot. However, Code LLMs lack repository specific knowledge such as the libraries used, unique architecture and coding style. For that reason, they may fail to provide accurate code suggestions.

Repository- level completion was addressed in [28], which refers to accurately completing the code in repositories. It is based on selective Retrieval Augmented Generation (RAG) where the system decides whether the code LM's generation could benefit from retrieved contexts or not. Another recent LLM-based repository code completion approach is GraphCoder [29]. The GraphCoder employs a graph-based retrieval-augmented process that leverages structural code context through a Code Context Graph (CCG) and locates context-similar code snippets. A prompt is then generated combining the context and the retrieved results and fed to LLM to return a predicted statement. Another approach that also uses graph representation is presented in [30]. It integrates a retrieval model that searches for similar code graphs to generate graph nodes, and a completion model based on a Multi-field Graph Attention Block.

Repository-level completion was also addressed in [31]. It leverages semantic information across files and utilizes pretrained code LMs to suggest the next code tokens. COCOMIC [32] leverages cross-file context and integrates it with the current file context in order to improve the output of pretrained code LMs for code completion.

User feedback information is leveraged in [9] for providing personalized code completion recommendations. An LSTM model is employed along with a fine-tuned BERT model to re-rank suggestions and generate more tailored code recommendations.

Finally, an approach for assisting code completion based on IDE usage logs is presented in [33]. Specifically, a pipeline for collecting anonymous usage logs from users is proposed. The usage logs are then used to train a decision tree for ranking code suggestions.

Regarding the research question RQ2 for code completion recommender systems, Table 4 summarizes the input that each recommender system requires from the user.

Table 4. Input of recommender system—code completion (RQ2).

Ref.	Input
[9,24]	Current code segment (token-based representation)
[28]	Current file context
[29,30]	Current code segment (graph representation)
[31,32]	Cross file context
[33]	Current code segment and usage logs

The recommenders described in [9,24] take as input the current code segment in token-based representation. As shown in Table 4, the recommenders in [29,30] use a graph-based representation of the current code segment. The work in [28] leverages current file context, while the recommenders in [31,32] utilize also cross-file context. Finally, usage logs along with the current user code are leveraged in [33].

As is described in Figure 6, 50% of the studies leverage the current code segment, while 25% use cross-file context.

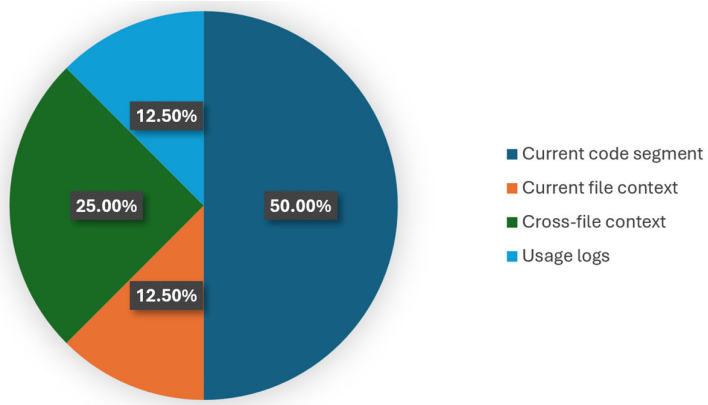

Figure 6. User input for code completion (RQ2).

Most of the code completion recommender systems suggest the next code token to use [9,24,32,33]. The recommender system presented in [28,30,31] suggests the next code snippets, while the one in [29] recommends a complete statement. Table 5 summarizes the results. Most of the publications recommend the next code token with the user types as it is depicted in Figure 7.

Table 5. Output of recommender system—code completion (RQ3).

Ref.	Output
[9,24,32,33]	Next code token
[29]	Next code statement
[28,30,31]	Next code snippets

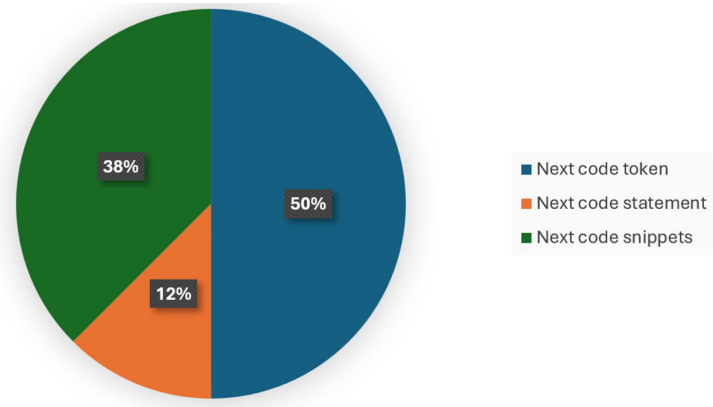

Figure 7. User output for code completion (RQ3).

Code completion recommenders generate recommendations based on AI models. Table 6 summarizes the methods used. The recommender system presented in [24] deploys transformer-based models to generate recommendations. LSTM and pretrained model BERT are used in [9]. Pretrained LLMS are employed also in [29,32]. Pretrained LLMs are large language models (such as GPT-4, BERT) that are pretrained on a large amount of data. RAG refers to the process of augmenting the LLMs' responses by incorporating external information. Authors in [28,31] employ RAG solutions for code completion. Finally, a graph-matching approach is presented in [30] while the recommender system in [33] uses a decision tree.

Table 6. Methods for generating recommendations—code completion (RQ4).

Ref.	Method
[9]	LSTM and pre-trained model BERT
[24]	Transformer-based model
[28]	Selective RAG
[29]	Coarse-to-fine retrieval process and LLM
[30]	Graph matching and Multi-field Graph Attention Block
[31]	Retrieval Augmented-Generation (RAG) solution
[32]	Pretrained Code LMs
[33]	Decision tree

Code completion approaches mainly are based on the current code segment. Some of them incorporate cross-file context to provide more relevant suggestions. However, they do not leverage specific user characteristics like, for example, a history of interactions. On the contrary, the recommender system described in [30] considers the developer's cloning behavior in order to recommend the completion code. Finally, usage logs are considered in [33].

4.3. Code Repair Recommendation

Recommender systems that suggest code repairs are included in this category. Those systems recommend bug fixes, refactoring code changes or refactoring. Recent studies [34,35] indicate a strong preference among developers to remain involved in the repair process by manually crafting patches, rather than relying on fully automated solutions [36]. VulAdvisor was proposed to facilitate developers repairing software vulnerabilities by automatically generating suggestions for it in the form of natural language.

Mahajan et al. [37] automatically recommend a Stack Overflow (SO) post for resolving a given Java Runtime Exception. An Abstract Program Graph (APG) is employed to represent the current code segment with the code snippets in SO posts and similarity matching is then applied to find the best match.

To assist developers in handling exceptions, the authors in [38] developed an Android Studio plugin, which recommends code for resolving runtime exceptions before they occur. It employs fuzzy logic for predicting the appearance of runtime exceptions and suggesting a related code to prevent them from happening.

A recurrent neural network-based method was presented in [39] that suggests which files to modify according to the developer's history of interaction. A code refactoring context-aware recommendation approach was proposed in [40]. Specifically, it exploits naming conventions, static contexts of the field and dynamic contexts of field renaming, which refer to field renaming conducted recently within the enclosing project to recommend field renaming.

Code change recommendations were proposed in [41] based on a neural network trained on tree code representation. A code refactoring recommender system was described in [42], which suggests refactoring based on feature requests. Finally, a code recommendation approach that suggests type conversion sequences based on coding context was proposed in [43]. It is based on a reachability analysis (i.e., analysis of whether a type can be converted to a target one) and semantic reasoning based on an ontology model.

Most of the RSSEs that recommend repairs take as input the current code segment [36–38,41], as it is shown in Table 7. As shown in Figure 8, 50% of the recommender systems for code repairs in this survey leverage the current code segment. The developer's history of interaction is considered in [39]. The RSSE in [40] takes as user input the field name to be renamed, while the RSSE in [42] takes as user input feature requests.

Table 7. Recommended code repairs—input of recommender systems (RQ2).

Ref.	Input
[36–38,41]	Current code segment
[39]	Developer's history of interactions
[40]	Field name to be renamed
[42]	Feature requests
[43]	Code and libraries of the current project

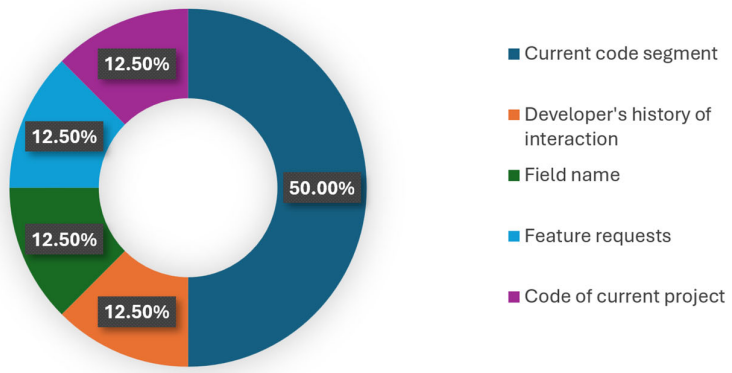

Figure 8. User input for code repairs (RQ2).

Regarding the output that the RSSEs present to the user (RQ3), there is a great variety in this category in comparison with the code recommendation and code completions. Table 8 presents in brief the output for each of the approaches. Code snippets are generated as recommendation output in [38] and [41]. The RSSE in [36] generates recommendations in natural language form. A related Stack Overflow post is recommended in [37]. The RSSEs in [39] recommend files to edit, while the RSSE in [42] recommends refactoring types. As depicted in Figure 9, 25% of the recommenders of this category suggest code snippets while the output of the rest varies based on their specific repairing purpose.

Table 8. Recommended code repairs—output of recommender systems (RQ3).

Ref.	Output
[36]	Natural language suggestions
[37]	Stack Overflow (SO) post
[38,41]	Code snippets
[39]	Files to edit
[40]	Field renaming
[42]	Refactoring types
[43]	Type conversion sequence

The methods used for generating recommendations are presented in Table 9. Fuzzy logic is utilized in [38], RNNs in [39] and graph matching in [37]. Similarity matching is used in [36] and sequence of context-aware heuristics in [40]. The authors in [41] implemented a tree-based hierarchical model while a Multinomial Naive Bayes (MNB) classifier is built in [42]. Finally, the research in [43] employs semantic ontology reasoning.

Regarding the use of user-specific context, only RSSEs in [39,42] consider the user-specific context to some extent. Specifically, the research in [39] considers the history of developer's interactions. The RSSE presented in [42] leverages the history of feature requests, code smell information, and the applied refactoring on the respective commits.

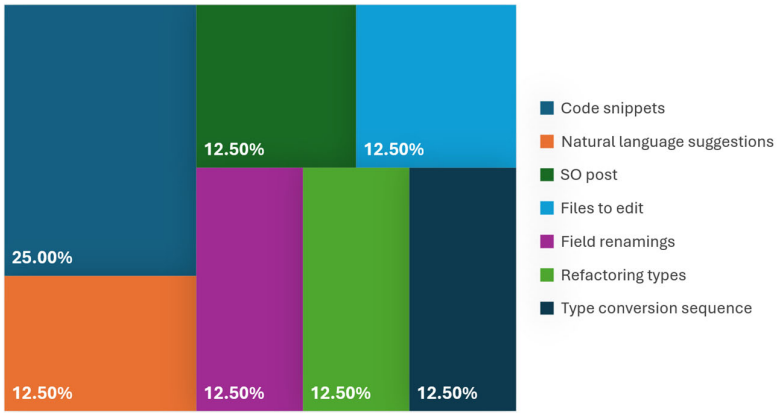

Figure 9. User output for code repairs (RQ3).

Table 9. Methods used for generating the recommendations—code repairs (RQ4).

Ref.	Method
[36]	Similarity matching
[37]	Graph matching
[38]	Fuzzy logic
[39]	Recurrent neural network
[40]	Sequence of context-aware heuristics
[41]	Tree-based neural network
[42]	MNB classifier
[43]	Semantic ontology reasoning rules

4.4. API Recommendation

API recommendation systems assist developers with finding the right API to use and correctly using it in their code. Several API recommenders generate recommendations based on a user query in natural language. An API query-based recommendation approach that considers user feedback was presented in [44]. Learning-to-rank and active learning were used to generate recommendations based on extracted features. The RSSE presented in [45] recommends APIs based on user natural language queries. It leverages the API usage in similar apps and employs similarity matching to find relevant APIs and present them to the developer. Another query-based API recommender is presented in [46]. It is based on tensor factorization and incorporates context information.

Another recommender system that generates recommendations according to the user's query was described in [47]. It relies on an evolutionary algorithm for optimizing API recommendations according to structural and semantic information extracted from a small dataset. An API recommendation approach that expands user query by retrieving related SO posts was proposed in [48]. The authors in [49] proposed CLEAR that employs the BERT language model and contrastive learning for enhanced representation of the user queries and recommends APIs that are more similar to them.

The RSSE presented in [50] takes a video or gif as input and recommends relevant APIs for creating the animation contained in the input. It employs a 3D CNN and GRU trained in data collected after analyzing APK files of apps. Similarly, the RSSE in [51] takes a video or GIF as input and finds relevant APIs based on similarity matching of temporal and spatial feature vector representations.

The vast majority of the API recommendations consider the current code segment to generate recommendations. An API recommendation approach for smart contract development was presented in [52]. It utilizes Graph Attention Networks and multilayer perceptron trained on AST (Abstract Syntax Trees), incorporating control and data flow

relations between and within statements in the smart contract code. An API recommendation technique for the Industrial Internet of Things (IIoT) was presented in [53]. It is based on the Matrix Factorization (MF) model and adds regularization terms to fuse user similarity and item similarity. Then, these two models are combined via linear combination to generate the final recommendation model.

An API recommendation system based on the hybrid of CF techniques was presented in [54]. It first applies a memory-based CF technique for identifying the most relevant projects by forming a rating matrix. Then, a model-based CF technique is employed to complete the matrix and refine the recommendation list.

The API recommendation system described in [55] encodes user code using pretrained code LMs like CodeBERT and CoT5 and identifies relevant APIs with similarity matching based on dot product.

Another API recommendation approach that uses a pretrained code LM to extract contextual information from source code was presented in [56]. A transformer-based model that was trained on a Python code dataset is used to predict the API that matches the code context.

The API recommender described in [57] employs the PDG (Program Dependence Graph) for code representation. A graph neural network (GNN) is employed to learn structure information and LSTM for learning text information, respectively. The outputs of GNN and LSTM are combined and fed to a Deep Neural Network (DNN) to recommend the most related APIs.

An LSTM-based API recommendation, namely Pythia, was presented in [58]. Pythia generates a ranked list of methods and API recommendations based on the developer's code leveraging abstract syntax trees. PyReco [59] leverages AST and tracks API usages to represent code and recommend APIs using a Nearest Neighbor classifier trained on open-source projects.

Most of the RSSEs for APIs assist users in finding the correct API. However, developers need assistance to use APIs effectively in their code. To this end, the recommender described in [60] leverages CF to suggest appropriate API function calls and provide related code snippets. The authors in [61] proposed an API recommender that uses call graphs to represent the hierarchical context and an inference model to recommend the most related methods to user code context. APIMatchmaker [62] recommends APIs and usages based on collaborative filtering, which takes app descriptions into account. Similarity matching is also used to refine the recommendation list. WebAPIRec was presented in [63], which encourages APIs to develop a project based on a personalized ranking model taking as input textual description and keywords that describe the project.

An API recommendation system was proposed in [64], which constructs three types of graphs to represent relationships between methods and APIs, APIs frequently used together and project structure. Attention networks are then employed to learn from graph representations and recommend related APIs.

An API argument recommendation approach can be found in [65], which suggests arguments based on code context. It integrates program analysis (PA) and language models (LMs) for suggesting relevant API arguments. Finally, a context-aware CF method [66] is applied to suggest relevant API calls and usage patterns. Table 10 summarizes the type of user input the RSSEs require in order to generate recommendations.

As depicted in Figure 10, almost half of the studies included in this survey and in the API recommendation category leverage the current code segment to generate recommendations. User query in natural language is required by 17.4% of the studies, while the 4.35% leverage both the natural language query and current code segment.

Table 10. API recommendation—input of recommender system (RQ2).

Ref.	Input
[44]	Natural language query and user feedback
[45,47–49]	Natural language query
[46]	Natural language query and current code segment
[50,51]	GIF or video file
[52,55–59,61,64–66]	Current code segment
[53]	User–API interactions, user–user similarity, item–item similarity
[54,60,62]	Current code segment and project
[63]	Project profile (description and keywords)

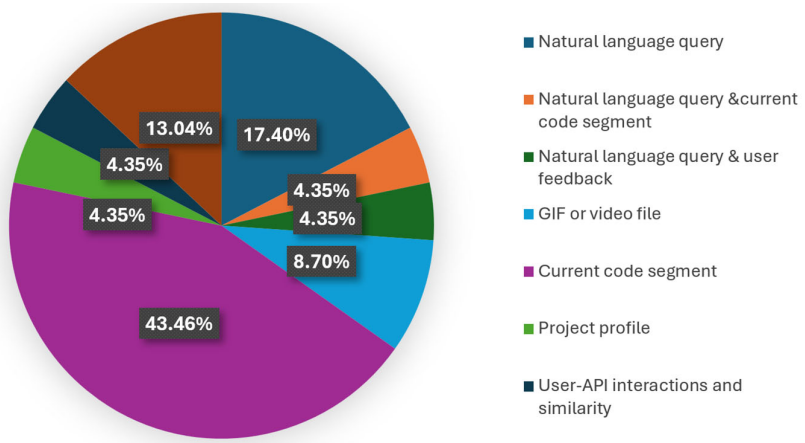

Figure 10. User input for API recommendation (RQ2).

A percentage of 73.9% of RSSEs for APIs generate a list of APIs based on user input/context as shown in Figure 11. Some of them also provide related code snippets and examples [60,62]. Table 11 summarizes the output of RSSEs for APIs.

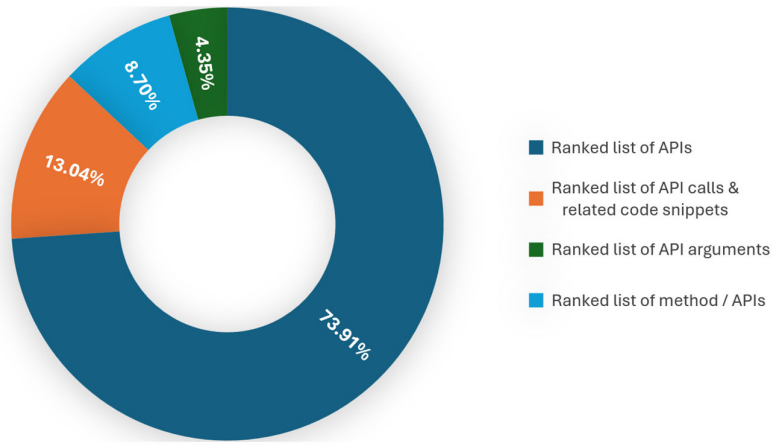

Figure 11. User output for API recommendation (RQ3).

Table 11. API Recommendation—output of recommender system (RQ3).

Ref.	Output
[44–57,61,63,64]	Ranked list of APIs
[60,62,66]	Ranked list of API calls and related code snippets
[65]	Ranked list of API arguments
[58,59]	Ranked list of methods/APIs

As it is described in Table 12, about one third of the examined API recommender systems are based on deep learning. One third is based on some form of similarity matching and three out of twenty-five are based on CF techniques.

Table 12. API recommendation—method of recommender system (RQ4).

Ref.	Method
[44]	Learning-to-rank and active learning techniques
[45,51,62]	Similarity matching
[46]	Similarity matching with tensor factorization
[47]	Optimization using a genetic algorithm for structural and semantic similarity
[48]	Similarity matching with Stack Overflow posts and API documentation
[49]	Contrastive learning with BERT embeddings and similarity matching
[50]	Multimodal deep learning
[52]	Graph Attention Networks and Multilayer Perceptron
[53]	Matrix Factorization combined with user and API similarity matching
[54]	Memory and model-based CF
[55]	Pretrained models and similarity matching
[56]	Transformer-based pre-trained model for feature extraction and deep neural network
[57]	Graph Attention Network, LSTM and deep neural network
[58]	LSTM based on AST representation
[59]	Nearest neighbor based on usage data
[60]	Context-aware CF (similarity matching)
[61]	Hierarchical context extraction and inference
[63]	Personalized ranking model
[64]	Heterogenous graph-based representation and attention networks
[65]	Ensemble ranking based on program analysis and language models
[66]	Context-aware CF

Regarding RQ5 and whether user context is considered, the recommender in [44] takes into account user selection of the recommended APIs and the recommender in [53] considers user–API interactions, and users' similarity.

4.5. Evaluation of RSSEs for Programming

To assess the ability of RSSEs to generate accurate recommendations, the authors of the reviewed papers conducted experimental evaluations. Evaluations mainly involved the application of the proposed methods on test data and measuring the accuracy of the recommendations against baseline methods. For that purpose, evaluation metrics are deployed. A common metric used by the majority of the RSSEs is Success Rate@K, which is defined as the proportion of correct recommendations as shown in Equation (1). If there is a match among the top-K recommendations, then the recommendation is considered correct. The same metric can be found in some works referred to as Hit@K or Top-K Accuracy or Accuracy@K. In the following equations, we use the word "query" as a general term to describe the request to the RSSEs to generate recommendations.

$$\text{Success Rate@K} = \frac{\text{Count of correct queries}}{\text{Count of queries}} \tag{1}$$

Another common evaluation metric is Precision@K (Equation (2)), which counts the number of correct recommendations in the top-K per query and averages with the number of

queries. In contrast with Success Rate@K, it takes into account the number of successful recommendations among the top-K list. Recall@K (Equation (3)) counts the number of correct recommendations in the top-K list divided by the ground truth recommendations.

$$\text{Precision@K} = \frac{\frac{1}{K}\sum \text{Count of correct recommendations}}{\text{Count of queries}} \tag{2}$$

$$\text{Recall@K} = \frac{\frac{\sum \text{Count of correct recommendations}}{\text{Count of relevant recommentdations (ground truths)}}}{\text{Count of queries}} \tag{3}$$

Another common metric used by the majority of the RSSEs to evaluate their recommendation accuracy is Mean Reciprocal Rank (MRR). The term reciprocal rank refers to the inverse rank of the first correct match in a recommendation list. MRR is computed as the average of the reciprocal ranks of all queries (N = the number of all queries):

$$\text{MRR} = \frac{\sum_{i=1}^{N} \frac{1}{\text{rank}_i}}{N} \tag{4}$$

Despite the use of those common evaluation metrics, there are also other evaluation metrics used that are more specific to the nature of the specific RSSEs. In particular, some code completion RSSEs use metrics such as Exact Match (EM) and Edit Similarity (ES) that measure the similarity of the recommended code with the ground truth. EM for a recommendation is either 0 or 1. If the recommended code matches the query, it is 1, and if it does not match, it is 0. For a set of recommendation queries, the EM is calculated as the average of the separate EM_i:

$$\text{EM} = \frac{\sum_{i=1}^{N} EM_i}{N} \tag{5}$$

Edit Similarity (ES) is less strict than EM since it measures how close the recommended code is to the ground truth. It is based on the Levenshtein distance, which measures how many edits are required to change one string to another. Given a test set, the ES is calculated as the average of the ES_i for each query of the test set as shown in Equation (6). ES_i is calculated by Equation (7), where y′ and y refer to the recommended code and the ground truth, respectively:

$$\text{ES} = \frac{\sum_{i=1}^{N} ES_i}{N} \tag{6}$$

$$ES_i = 1 - \frac{\text{Lev}(y_i, y_i')}{\max(|y_i|, |y_i'|)} \tag{7}$$

Table 13 summarizes the evaluation process for the reviewed papers including evaluation metrics, datasets, compared methods and results. The authors in [12,50,60] conducted additional user evaluation studies. The aim of most of these studies was to measure user satisfaction and the perceived usefulness. The participants were asked to use the systems and then they were asked various questions, such as whether they found the recommendations useful. The Lickert scale (1–5) is a common metric for that purpose. Average completion time was also measured in [50,60], which referred to the time needed by the users to complete a code task with and without the assistance of the RSSEs.

Table 13. Evaluation metrics, datasets, results, remarks and benchmarking of different RSSEs.

Ref.	Evaluation Metrics	Datasets	Results/Comparative Results	Remarks/Benchmarking
[11]	Recall@K (K = 1, 3, 5, 10), Precision@K (K = 5, 10), NDCG@K (K = 5, 10)	BigCloneBench, 743 open-source Java projects from GitHub (741,148 code snippets)	Recall@10 of 88.7% and Recall@1 of 37.3%	Outperformed all compared methods
[12]	Precision@100, Recall@100, F1@100, Query Time (s), Usefulness (Likert 1–5)	CodeSearchNet and Neural Code Search	P@100 of 92.50% and F@100 of 42.95 at CodeSearchNet. P@100 of 68.83% and F1@100 of 56.42% at Neural Code Search. 147.9× faster than Aroma on CodeSearchNet and 224× faster on Neural Code Search. MinHash is faster on Neural Code Search but Senatus has comparable time and 10x Precision, Recall and F-measure	Outperformed compared methods
[13]	Perplexity (PPL), MRR, Top-K Accuracy (K = 1, 3, 5, 10)	BigCloneBench, IJaDataset	MRR of 29%, Accuracy Top-1 of 23.8%, Top-3 of 32.5%, Top-5 of 36.2% and Top-10 of 40.5% for Exact Match and MRR of 74%, Accuracy Top-1 69.4%, Accuracy Top-3 77%, Accuracy Top-5 of 80.1% and Accuracy Top-10 of 84.5%	Comparing perplexities for Deep Clone and Clone advisor, where Clone advisor showed lower perplexities in top-10 retrieved snippets
[14]	MRR, Hit@K (K = 1, 3, 5)	CodeSearchNet	MRR 0.44, Hit@1 of 36.75%, and Hit@5 of 57.35%	Outperformed compared methods
[15]	SuccessRate@K (K = 1, 5, 10), MRR	Stack Overflow (Top 100 questions), GitHub Java projects	SuccessRate@1 of 32%, SuccessRate@10 of 57% and MRR 0.44	Outperformed baselines
[16]	Precision	Tested only with three sample queries	Precision 70–86.67% for 3 sample queries	-
[17]	Precision@K (P@5), Success Rate, Wilcoxon Test	120 Android Java projects from GitHub (Music Player, Bluetooth Chat, Weather, File Management)	P@5 of 94% and success rate of 90–95%	Outperformed FACER
[20]	Accuracy@K (K = 1, 10) for incomplete code, Classification Accuracy for complete code	Online Judge (OJ) System dataset (52,000 files from 104 programming problems)	Accuracy@10 of 97.49%	Outperformed compared methods

Table 13. *Cont.*

Ref.	Evaluation Metrics	Datasets	Results/Comparative Results	Remarks/Benchmarking
[21]	Top-1 Accuracy, Top-3 Accuracy	14,807 Java projects (350 M lines of code, 2 M files), evaluated on 10 large Java projects with 23,000 to 400,000 commits	Top-1 Accuracy of 66% and Top-3 Accuracy of 74%	Outperformed baselines
[22]	Precision@K (K = 1, 3, 10), Recall@K (K = 1, 3, 10)	JEdit, ArgoUML, JHotDraw (method parameter recommendations)	Precision@10 of 72.06% in Eclipse and 78.38% in NetBeans	Outperformed compared methods
[23]	NDCG@10, Quality Improvement Score	Java and Python code generated from 5 LLMs	Improved NDCG@10 score	Improved NDCG@10 score for all compared methods
[9]	Hit@K (K = 1, 3, 5, 10), MRR	Dataset provided by CodeGRU (https://github.com/yaxirhuxxain/Source-Code-Suggestion accessed on 21 March 2025), built from open-source Java Github project	Hit@1 0.4998, Hit@3 0.6319, Hit@5 0.6759, Hit@10 0.7191 and MRR 0.5764 when compared to N-gram and Hit@3 0.5986, Hit@5 0.7576, Hit@5 0.8056, Hit@10 0.8425 and MRR 0.6867 when compared with CodeGRU	EHOPE outperformed the baselines
[24]	Accuracy@K (K = 1, 3, 5, 10), MRR@K (K = 1, 3, 5, 10), Precision, Recall, F1-Score	Java and C# datasets collected from GitHub	Accuracy@10 of 90.10% (Java) and 86.05% (C#), MRR@10 is 75.13% (Java) and 68.66% (C#)	Outperforming all baselines. Precision and Recall surpass previous models.
[28]	Exact Match (EM), Edit Similarity (ES), Unit Test Pass Rate (UT)	RepoEval, CrossCodeEval, CrossCodeLongEval	EM 54.40 and ES 76.00 at line level, EM 46.10 and 72.70 ES at API-level and UT 28.79 and ES 57.30 at function level for RepoEval dataset	Outperformed methods compared in terms of EM, ES, UT in various experimentation settings.
[29]	Exact Match (EM), Identifier Match (IM)	8000 repository-level completion tasks from 20 repositories	Achieved higher EM	Achieved higher EM improved by +6.06 and IM by +6.23 over baselines
[30]	Accuracy (Value and Type), Precision, Recall, F1-Score	PY150K (Python), JS150K (JavaScript), PY1K, PY10K, PY50K, JS1K, JS10K, JS50K (Filtered vocabulary versions)	Achieved accuracy of 80.8% (JS1K) and 75.1% (PY1K)	Outperformed baselines.
[31]	Edit Similarity (ES), Identifier F1-Score (ID-F1), SpeedUp (%)	CrossCodeEval benchmark Python dataset	ES 80.82 and ID-F1 77.31	Outperformed compared variations achieving an improved speed of 33.29% and 48.41% for prompt Length of 2048 and 1024

Table 13. *Cont.*

Ref.	Evaluation Metrics	Datasets	Results/Comparative Results	Remarks/Benchmarking
[32]	Exact Match (EM), BLEU-4 for code match, EM, Precision and Recall for Identifier Match, Perplexity (PPL)	60891 projects from Python Package Index	+33.94% improvement in Exact Match (EM), +28.69% improvement	COCOMIC outperformed in ID Match over in-file-only baselines
[33]	Recall@K (K = 1, 5) for offline evaluation and for online evaluation with user-defined session-based metrics (i.e., Explicit Select Rate, Typed Select Rate, Typing Actions, Prefix Length, Manual Start Rate)	Usage logs collected from Python projects in PyCharm for 2 weeks	Improved Recall@1 from 0.761 to 0.870, Recall@5 from 0.957 to 0.981	Outperformed baseline in all metrics and settings
[36]	BLEU, ROUGE-L, BERTScore, RAS	Dataset of 18,517 pairs of vulnerabilities and suggestions from open-source projects	BLEU 21, ROUGE-L 34.7, BERTSCORE 67.7 AND RAS 12.5	Outperformed all compared methods
[37]	I-score (percentage of perfect SO posts, IH-score (percentage of relevant SO posts) and M-score (percentage of irrelevant posts)	Dataset based on Stack Overflow dump and top 500 open-source Java repositories in Github	0.40 I-score, 0.71 IH-score and 0.26 M-score	Outperformed compared methods
[38]	Top-K Accuracy (K = 1, 3), percentage of fixes of developers/recommendations	Dataset constructed by crawling apps and collecting exception bugs from open-source repositories resulting in 1000-exception bug dataset	Top-1 accuracy 73–75% for correct warning on exceptions (different setting of risk level). Similarly, Top-3 accuracy 79–81%	65% of the recommendations were applied by developers, where 21% were higher than CAR-Miner and 37% higher than heuristic-based recommendation
[39]	Precision, Recall, F1-Score	Interaction traces data collected by Eclipse Bugzilla with Mylyn plugin	Average F-1 score 0.64 vs. 0.59	Performed slightly better than MI-EA when recommendations stop after the first incorrect edit
[40]	Precision, Recall, F1-Score	11,085 real-world field renamings collected from 388 open-source Java projects with RefactoringMiner	49.44% F1 score while IDEA scored 6.3%, Incoder 13.41% and Zhang's test 20.17%	Outperformed compared methods for all metrics.
[41]	Top-K Accuracy (K = 1, 2, 5)	Code-Change-Data, Pull-Request-Data, Defects4J-data	15.94% Top-5 Accuracy for Code-Change-Data and 28.87% for Pull-Request-Data	Outperformed compared methods.

Table 13. *Cont.*

Ref.	Evaluation Metrics	Datasets	Results/Comparative Results	Remarks/Benchmarking
[42]	Accuracy, Precision, Recall, F1-score, Hamming Loss, Hamming Score	Dataset from 55 open-source Java repositories and 18,899 feature requests from JIRA issue tracker	Precision 76% vs. 20%, Recall 54% vs. 34% and F-measure 61% vs. 25%	Significantly outperformed the compared method for all the evaluation metrics
[43]	Hit Rate @ K (K = 3, 10),	Tomcat 7.0.47 source code, 1338 code snippets requiring type conversion sequences containing 145 static method entry points	72.2% top-3 hit rate vs. 60.7% and 90.3% top-10 hit rates vs. 78.4%	Outperformed Eclipse Code recommenders
[44]	Hit@k (Top-k Accuracy), MAP (Mean Average Precision), MRR (Mean Reciprocal Rank)	BIKER, RACK, and NLP2API datasets	For 100% accumulation of repository Hit-1, values improved by 9.44% for BIKER (method level), 6.79% for BIKER (class level), 18% for RACK, and 18.39% for NLP2API	Improved baseline methods for all metrics as accumulation of the feedback repository increased.
[45]	Precision@N(P@N) and Mean Average Precision@N(MAP@N)	Dataset made by crawling Google Play Store apps from 4 categories (rating $\geq$4.5)	Precision@4 of 0.49, Precision@5 of 0.53, Precision@10 of 0.69 and MAP@1 of 0.31, MAP@5 of 0.34 and MAP@10 of 0.34	Showed higher results than compared method, but they are not comparable since they were not based on the same dataset
[46]	SuccessRate@N, Precision@N, Recall@N, Mean Average Precision (MAP@N), Mean Reciprocal Rank (MRR@N), Normalized Discounted Cumulative Gain (NDCG@N)	Official data dump of StackOverflow, 125,847 Java questions, 62,067 (query, API, context) triplets extracted, 458 test queries manually constructed. Test dataset used in BIKER [16]	Outperformed BIKER, SuccessRate@1 of 39.5% vs. 30.0%	Outperformed significantly RACK, higher values for all metrics (10–45% improvements)
[47]	Precision@N (P@N), Mean Average Precision@N (MAP@N), Mean Reciprocal Rank (MRR)	Google Play apps in 5 categories: Android API descriptions and Q&A from Stack Overflow (for ground truth)	Outperformed LibraryGuru in all metrics improvements for 19.4% to 91.7%. Similarly, outperformed GAPI improving metrics ranging from 106.3% to 1050%	Outperformed LibraryGuru and GAPI
[48]	MRR (Mean Reciprocal Rank) and MAP (Mean Average Precision), Success Rate (S @ K)	Stack Overflow Data, 413 manually labeled test queries	Improved S@1 by 27.2% over BIKER and by 22.3 over BRAID at method level. Similarly, improved S@1 by 22.3% over BIKER and by 24 over BRAID at class level	Outperformed compared methods
[49]	MRR (Mean Reciprocal Rank), MAP (Mean Average Precision), Precision@N, Recall@N	Stack Overflow data from BIKER dataset	Improved Precision@1 by 314.94% to 732.24% and Recall@1 by 133.18% to 326.29%	Outperformed compared methods at method and class level

Table 13. *Cont.*

Ref.	Evaluation Metrics	Datasets	Results/Comparative Results	Remarks/Benchmarking
[50]	Accuracy@N (for N = 1, 3, 5, 10), Average Completion Time in user study	960 apps from Google Play apps from 4 categories, resulting in 5329 mappings between UI animations and API sequences	Average Accuracy@1: 45.13% and Accuracy@10: 81.85%	Outperformed LUPE; however they are not exactly comparable methods
[51]	Success Rate@N, MAP@N (Mean Average Precision), Precision@N, Recall@N	Top-20 free Android apps from 32 categories of Google Play, resulting in 3200 animation-API mappings. Rico dataset	Precision@20 230.77% and Recall@20 improvement by 184.95%	Outperformed Guru
[52]	Accuracy@N (Top-1, Top-2, Top-3, Top-5, Top-10), MRR (Mean Reciprocal Rank)	Collected 25,000 Solidity smart contract projects from Etherscan	Top-1 Accuracy of 64.85% (214.19% higher than best baseline). Top-5 Accuracy: 71.65% (81.43% higher than best baseline). MRR: 68.02% (106.4% higher than best baseline)	Outperformed baselines
[53]	MAE (Mean Absolute Error), RMSE (Root Mean Square Error)	Crawled 17,412 APIs from ProgrammableWeb	Best MAE = 0.151, RMSE = 0.204 for 90% training set density	Outperformed all compared methods
[54]	Success Rate@N, Precision@N, Recall@N, MRR (Mean Reciprocal Rank), NDCG (Normalized Discounted Cumulative Gain) (N = 1, 3, 5)	SHL (610 Java projects from GitHub), SHS (200 Java projects from SHL), MVL (3600 JAR archives from Maven Central Repository), MVS (1600 selected unique projects from MVL)	-	Outperformed FOCUS in most cases; it performed better in small and sparse datasets. FOCUS performed slightly better in large datasets
[55]	Top-K Accuracy (Top-1, Top-5, Top-10), MRR (Mean Reciprocal Rank)	APIBench dataset, Java and Android APIs	Top-1 Accuracy 77.37%, Top-5 94.79%, Top-10 98.15% and MRR 0.851 in Java dataset and Accuracy Top-1 71.60%, Top-5 90.21%, Top-10 94.46% and MRR 0.798 in Android	Outperformed all baseline approaches
[56]	Top-K Accuracy (k = 1, 2, 3, 4, 5, 10), MRR (Mean Reciprocal Rank)	Intra-Project Edition (constructed using 8 Python open-source projects from GitHub)	Average Top-1 Accuracy 40.27%, Top-2 Accuracy 45.15, Top-3 Accuracy 49.44, Top-4 Accuracy 52.83, Top-5 Accuracy 54.18, Top-10 Accuracy 60.38 and MRR 47.30%	Outperformed compared methods.

Table 13. *Cont.*

Ref.	Evaluation Metrics	Datasets	Results/Comparative Results	Remarks/Benchmarking
[57]	Top-K Accuracy (Top-1, Top-5, Top-10)	625 Java projects from GitHub, over 1000 stars and datasets used by the compared approaches	Top-1 Accuracy 67.3–70.1%, Accuracy Top-5 85.1–90.8%, Top-10 91.3–95.8%.	Outperformed the compared methods in most of the datasets. However, the compared methods were not reproduced and their results were taken directly from their papers.
[58]	Top-K Accuracy (Top-1, Top-5), MRR (Mean Reciprocal Rank)	2700 Python open-source GitHub projects, 15.8 million method calls	Top-1 Accuracy 0.71, Top-5 Accuracy 0.92 and MRR 0.814	Outperformed all compared methods.
[59]	MRR (Mean Reciprocal Rank), Recall	20 Python libraries	Achieved average MRR 0.5 and recall 0.84	Outperformed significantly compared methods.
[60]	Success Rate@N, Precision@N, Recall@N, Levenshtein Distance, Time, User-Perceived Usefulness	26,854 API functions extracted from 2600 open-source Android apps (from Google Play and GitHub)	Success rate of 92.10% vs. 58.40% of PAM and 40.66% UP-Miner. UP-Miner is the fastest (regarding recommendation time). FOCUS is faster than PAM	FOCUS outperformed significantly compared methods in all experiments. Majority of users evaluated positively (69%) that recommendations are relevant
[61]	Top-K Accuracy (Top-1, Top-5, Top-10), Execution Time	Datasets used by compared methods, galaxy, log4j, spring, antlr, jGit, froyo-email, grid-sphere and itext	-	HiRec outperformed methods compared with top-5 and top-10 accuracy rates. APIREC is close to HiRec regarding top-1 accuracy.
[62]	Success Rate@N, Precision@N, Recall@N	12000 Android apps	-	APIMatchmaker outperformed both FOCUS and statistics baseline in terms of success rate, precision, and recall.
[63]	Hit@N, MAP@N, MRR (Mean Reciprocal Rank)	9883 web APIs and 4315 projects from ProgrammableWeb	Hit@5, Hit@10, MAP@5, MAP@10, MAP, and MRR scores of 0.840, 0.880, 0.697, 0.687, 0.626, and 0.750, respectively.	Outperformed compared methods

Table 13. *Cont.*

Ref.	Evaluation Metrics	Datasets	Results/Comparative Results	Remarks/Benchmarking
[64]	Success Rate@K, Precision@K and Recall@K (K = 1, 5, 10, 20). User study (6 developers): Relevance and preference	SHS (200 Java projects), SHL (610 Java projects), MV (868 JAR archives)	SR@1 0.439, SR@5 0.672, SR@10 0.794 and SR@20 0.836 for MV dataset, while the best results of comparisons were for GAPI SR@1 0.195, SR@5 0.363, SR@10 0.479 and SR@20 0.600	Outperformed compared methods in all datasets and for all evaluation metrics
[65]	Top-K Accuracy (K = 1, 3, 5, 10), Precision, Recall, MRR (Mean Reciprocal Rank)	Small corpus: 2 large projects, Eclipse and Netbeans Large corpus: 9271 projects	The NetBeans dataset achieved MRR 0.72 while GPT-2's MRR was 0.55, CodeT5 MRRS was 0.63 and SLP's MRR was 0.44	Outperformed all baselines for all metrics and datasets
[66]	Success Rate@N, Precision@N, Recall@N, Recommendation Time	610 Java projects from GitHub, 200 small Java projects, 3600 JAR archives from Maven Central	SuccessRate@1 24.59% for a small dataset vs. SuccessRate@1 72.30 a larger dataset. FOCUS is about 100× faster than PAM (average recommendation time 0.095 s vs. 9 s)	Outperformed compared method. Achieved better evaluation metrics in large datasets

5. Discussion

Recommenders for programming assist developers by providing related code and information while programming. Research in this area has made great advancements in recent years. In this review, four main categories of recommenders for programming were identified based on the way they help programmers: code recommendation, code completion, code repair recommendation and API recommendation (RQ1). Code recommendation and API recommendation categories seem to be areas of active research during the date range of our research (published after 2014). On the contrary, recommender systems on code completion have become more evident in recent years (after 2022). This is mainly due to the wide adoption of LLMs, which are applications for assisting programmers to write code.

The second question we aimed to answer with this survey was to identify what user input is required by those systems to perform recommendations. Over half of the recommender systems studied in this survey use the developers' code as user input in order to generate recommendations. Specifically, 36% of them use the current code segment of the user's code while 16% of them leverage code written in the whole file or/and related files and/or the complete project. In total, 22% of the reviewed approaches require user input in natural language.

Table 14 summarizes the results highlighting the different types of inputs (RQ2), outputs (RQ3) and methods (RQ4) used in recommenders for programmers. Regarding RQ3, it appears that most recommenders generate a list of recommendations or output the best-matched one. For example, most code recommender systems present a list of code snippets to the user. Similarly, API recommenders return a list of relevant APIs. It seems that there is not much focus on explaining why the specific recommendations are made. Explainability if used correctly can increase trust and transparency in recommender systems [67].

The methods used for generating recommendations vary and span from fuzzy logic to similarity and graph matching. However, it is evident that many of those systems deploy deep learning and/or LLMs either for extracting and representing features or for training classifiers (RQ3). Generally, it seems that the main challenge that these approaches face is how to better represent code context and match coding patterns learned from mining code repositories. In the following section, deep learning architectures deployed in the reviewed papers are discussed in detail, emphasizing their strengths and weaknesses.

Deep Learning Architectures

About half of the papers of this survey implemented deep learning-based approaches. This comes as no surprise since Neural Networks are very powerful when trained in vast amounts of data. Given the large and growing number of open-source code repositories, deep neural networks can be trained to learn coding patterns and leverage that knowledge for assisting in code-related tasks.

Transformer-based architectures are utilized in many works and especially for code completion. In seven out of eight of the code completion approaches included in this survey, pretrained LMs are used. LMs are based on the transformer architecture, which uses the self-attention mechanism that allows processing input in parallel. They are trained in large-scale repositories and can effectively generalize in tasks like code completion where the aim is to predict the next token. However, they cannot fully leverage projects and user-specific context. Fine tuning and RAG can assist in this direction; however, it can have a large computational cost.

Table 14. Summarization table of inputs, outputs methods used for recommender systems for programmers.

Aim (RQ1)	Input (RQ2)	Output (RQ3)	Method (RQ4)	Ref.
Code Recommendation	Current code segment, natural language query, methods with API usages in current active project, developers' coding history, project-specific and common code patterns	Ranked list of code snippets, ranked list of related methods, a list of potential functionalities, ranked list of code elements, method parameters	Self-attention neural networks, AST-based feature scoring and De-Skew LSH, pretrained GPT-2 and IR technique (TF-IDF), SBERT model and similarity matching, GRU Network and JEAN model, similarity matching, deep learning model Fuzzy sets, heuristics and ranking based on quality criteria applied in the output of transformer-based code generation model, similarity matching (cosine)	[11–17,20–23]
Code Completion	Current code segment (token-based representation), current file context, current code segment (graph representation), cross file context, current code segment and usage logs	Next code token, predicted statement, next code snippets	LSTM and pre-trained model BERT, transformer-based model Selective RAG, coarse-to-fine retrieval process and LLM, Graph Matching and Multi-field Graph Attention Block, Retrieval Augmented-Generation (RAG) solution	[9,24,28–33]
Recommending Code Repairs	Current code segment, current code segment and associated Java, runtime exception, developer's history of interactions, field name to be renamed, feature requests, code and libraries of the current project	Natural language suggestions, Stack Overflow (SO) post, code snippets, files to edit, field renaming, refactoring types, type conversion sequence	Similarity matching, graph matching, fuzzy logic, recurrent neural network, sequence of context-aware heuristics, Tree-based neural network, MNB classifier, semantic ontology reasoning rules	[36–43]

Table 14. *Cont.*

Aim (RQ1)	Input (RQ2)	Output (RQ3)	Method (RQ4)	Ref.
API Recommendation	Natural language query and user feedback, natural language query, natural language query and current code segment, GIF or video file, current code segment, user–API interactions, user–user similarity, item–item similarity, current code segment and project, project profile (description and keywords)	Ranked list of APIs, ranked list of API calls and related code snippets, ranked list of API arguments, ranked list of methods/APIs	Learning-to-rank and active learning techniques, similarity matching, similarity matching with tensor factorization, optimization using a genetic algorithm for structural and semantic similarity, similarity matching with Stack Overflow posts and API documentation, contrastive learning with BERT embeddings and similarity matching, multimodal deep learning, Graph Attention Networks and multilayer perceptron, matrix factorization combined with user and API similarity matching, memory and model-based CF, pretrained models and similarity matching, transformer-based pre-trained model for feature extraction and deep neural network, Graph Attention Network, LSTM and deep neural network, LSTM based on AST representation, nearest neighbor based on usage data, context-aware CF (similarity matching), hierarchical context extraction and inference, personalized ranking model, heterogenous graph-based representation and attention networks, ensemble ranking based on program analysis and language models, context-aware CF	[44–66]

Graph Neural Networks (GNNs) are deployed for leveraging the structure of the code and better modelling the developer's context. GNNs can capture complex relationships in code projects and cross file context. Therefore, they can lead to better representations and increased accuracy of recommendations. As shown in [29], the graph-based code representation improved repository-level code completion. Another advantage of graph-based approaches is that they are interpretable. Their structure can be used to provide explanations to developers for the generated recommendations. Despite the advantages of graph-based Neural Network architecture, there are some considerable drawbacks. Graph construction requires high memory usage leading to increased computational cost. Moreover, the accuracy of their recommendations depends on the quality of the graphs that are constructed. So, when there is not enough context to leverage those methods, they will not perform well, resulting in less accurate recommendations.

RNNs are less computationally intensive than Transformers and GNNs. They are deployed in cases when sequential information is crucial, like recommending code edits. However, as sequences become larger, their performance degrades. This is due to the Vanishing Gradient problem, which refers to the phenomenon of very small gradients being produced as information is propagated. Thus, they are more suitable when short dependencies are enough for the code task and there is no need to leverage large-scale dependencies. LSTMs are an improved type of RNN that uses memory cells to store and control information flow. This allows it to learn long-term dependencies better than traditional RNNs. The authors in [9] deployed LSTM for code completion, while the authors in [58] leverage LSTM for API recommendation. GRUs are another improved type of RNN that addresses the Vanishing Gradient problem using gates to control the information and learn using less parameters, making it more efficient in sequential tasks. Due to its simpler architecture, it is faster than LSTM. The RSSE presented in [15] deployed GRU along with an attention-based module for code recommendation showing increased accuracy of recommendations.

Regarding RQ5, only 7 out of 50 consider some kind of user context despite the current code or user query. As shown in Figure 12, 86% of the studies included in this survey do not leverage user-specific context and therefore do not provide personalized recommendations.

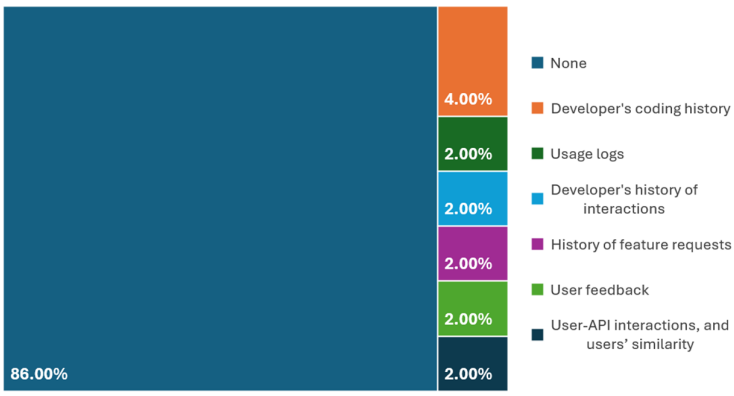

Figure 12. User context—personalization recommendation (RQ5).

The research presented in [21] could be considered as more focused on providing personalized recommendations. Personalization can increase the effectiveness of recommender systems, and it seems it is highly neglected in recommender systems for programmers. A recent study [68] highlighted the importance of personalization in source code recommenders and stated that being aware of the developers' knowledge (i.e., their expertise, past implementation tasks, etc.) could result in more relevant recommendations.

RSSEs for programmers mainly exploit current user code and/or a natural query given by the user to recommend relevant code based on coding patterns learned after training in large code repositories. This process is quite different from traditional Recommender approaches such as Content-based and Collaborative Filtering, which are user-centered.

The task-oriented way that RSSEs are implemented helps address the 'cold start' problem of traditional recommender approaches. More specifically, RSSEs can provide recommendations for new users since they mostly base the recommendation generation process on large repositories coded from various developers. However, the recommendations made in this way are the same for all programmers.

Programmers are individuals with specific characteristics that set them apart from the rest. For example, a programmer could be a professional with several years of experience, while another one could be a novice who just started to learn programming. These two programmers do not require the same type of assistance in their coding tasks. Thus, the same recommendations would not be suitable for both. So, regarding RQ6, this realization highlights a problematic topic in the current state of RSSEs, i.e., the lack of personalization in most RSSEs.

Proper user modelling that considers user-specific user context such as coding style, library preferences and level of experience could lead to more personalized recommendations. RSSEs could leverage traditional recommender approaches, namely Content-based and Collaborative Filtering, to form hybrid recommenders that utilize both good coding practices and user-specific context.

Furthermore, the adoption of the recommendations of programmers should be taken into consideration and used for refining the recommendations. When a recommendation is made, it should be reported whether it was used by the programmer or not. Then, this information could be utilized for refining suggestions. The authors in [9,44] exploited users' implicit feedback for code completion and API recommendations, respectively. Techniques like active learning and reinforcement learning can help to dynamically adapt the recommendations based on user feedback. This way, the recommendations generated will be tailored to the specific user, resulting in a more personalized experience.

Another problematic topic identified by this survey is the lack of explanations in recommendations. Programmers require assistance in their coding tasks, but want to be sure that the recommendations can help them achieve their goals regarding the coding task they work on. Thus, providing explanations for why the specific recommendations were made can help them understand if the recommendations are suitable for their purposes. This way, the system is more transparent to the users and leads to an increase in users' trust.

Many techniques can help to provide explainable recommendations. First, machine learning models that are interpretable can be utilized for providing explanations. For example, decision trees have a tree structure that can be used for justifying explanations. However, decision trees often lack accuracy compared to deep learning models. Deep learning models are generally referred to as 'black boxes' since their architecture is made of many layers and parameters, making it difficult to interpret them.

Model-agnostic methods can be used for interpreting DL models [69]. Well-known methods for that purpose are LIME [70] and SHAP [71]. LIME is based on creating a simple interpretable model that approximates the complex model, like a DL model, and uses this model's coefficients for explaining their contribution to the prediction. It is an easy and fast method that can be used for explaining individual recommendations.

SHAP is a more robust method that is based on Shapley values from game theory. SHAP provides a value of importance in each feature representing its contribution to prediction. Those values can be used to present visually the most important features, highlighting their relationship with the predicted recommendations. In this way, the

users can understand why the specific recommendation was made, making the system more transparent.

Even though DL models are generally considered as black boxes, GNNs can be leveraged for providing explanations due to their structure based on graphs. Studies like GNNExplainer [72] and GraphLIME [73] utilize the structure of GNNs to identify important features and relationships. Therefore, RSSEs that deploy GNNs can use their structure to justify their recommendations.

Another problematic topic identified is that only three papers stated that they conducted some type of user survey, and with a very small number of participants. Evaluating RSSEs with real users and measuring user satisfaction is crucial for successful RSSEs. Moreover, the improvements introduced using RSSEs in completing their coding tasks should be measured to assess the impact of the recommendations generated. For instance, the average time needed for completing a coding task with and without the use of RSSE is an indication of the impact of the RSSEs in completing a coding task. The quality of the generated code with or without the use of RSSEs could also be assessed to identify whether RSSEs had a positive impact on completing the coding task. Proper user evaluation can highlight areas of improvement and lead to better user experience and, therefore, increased user satisfaction.

To this end, regarding RQ7, RSSEs for programmers mostly focused on API recommendation during the first years of the last decade. On the contrary, in recent years (after 2022), there has been a growing interest in research for code completion tasks, as shown in Figure 3. This does not mean that code completion was not an important task to address before, but that it can now be addressed more efficiently due to the recent advances in DL. DL and especially the advances in LLMs allowed for higher accuracy in code recommendations. Therefore, in recent years, there has been a trend of utilizing DL architectures and LLMs for RSSEs. The number of research papers will keep growing as this is still the start of this new era in AI advancements. Thus, research in RSSEs is expected to increase in the near future and lead to improvements in the quality of recommendations and increased user satisfaction. Moreover, given that research on RSSEs mostly focused on the accuracy of recommendations in recent years, there may be a shift in focusing on improving user satisfaction by providing more personalized suggestions and explainable recommendations.

Based on the results of this survey, we foresee two main directions for future improvements, i.e., personalization and explainability. Techniques that identify software developers' programming styles, expertise, and specific libraries are of great importance and could be leveraged to tailor the recommendations in each developer and thus provide more personalized suggestions. Those techniques can be used for modelling developers' profiles representing user-specific characteristics such as coding style and expertise. User profile representations can then be utilized by RSSEs to generate more personalized suggestions. Developing hybrid RSSEs combining deep-learning based approaches with Content-based and Collaborative Filtering are worth exploring for code recommendation tasks.

Another future direction for providing more personalized recommendations is real-time adaptation to user feedback and changing user preferences. Allowing programmers to give explicit feedback on the recommendations can help refine the suggestions. Implicit feedback can also be given by keeping track if they accept the recommendations. This information can then be used to refine the recommendations. Reinforcement learning (RL) can also be utilized for adapting code recommendations based on user feedback. RL refines suggestions based on user interactions and coding history. This way, it leads to more personalized recommendations and thus increased user satisfaction.

Last but not least, RSSEs can leverage explainability techniques for justifying their recommendations. Explaining why the specific recommendations are made is crucial

in increasing transparency and thus making programmers trust the recommendations provided. If RSSEs provide recommendations without making it obvious why those recommendations are applied, it is difficult for the developers to trust them. Programmers need to feel in control. By making it transparent why specific recommendations are made, programmers can more easily decide if they are going to apply them or not.

6. Conclusions

Recommender systems for programming have seen great advancements in recent years in various tasks such as code recommendation, code completion, code repairing and API recommendation.

Most of those approaches take as user input the current code segment being edited, while there is also a handful of approaches that require user input in the form of natural language query. Based on the user input, the recommender systems generate suggestions in order to assist developers with writing code. AI methods, and mostly deep learning and pretrained code LMs, are used for to form classifiers trained in a vast amount of data from large code repositories. Nevertheless, there is a great variety of different methods employed for generating recommendations. Regardless of the method used, the main challenge of all those approaches is how to better represent the user's code, how to match with code patterns and how to learn code patterns in the first place.

The main focus of the majority of research efforts has been on the increase in the accuracy of recommendations. Most of them are based on source code context ignoring user-specific context, for example, frequently used libraries. Modelling user context and using it to provide more personalized recommendations could enhance user satisfaction.

Explainable recommendations are also highly dismissed in recent research efforts. Explainable recommendations help humans to understand why certain items are recommended. Explaining to programmers why the specific recommendations are made can increase trust in the recommender systems.

Given the above, future directions could include the development of more personalized and explainable recommender systems for programmers. Personalization and explainability could enhance developers' satisfaction and increase trust and transparency.

Author Contributions: Conceptualization, G.A.P.; methodology, E.M.; formal analysis, E.M.; investigation, E.M.; resources, E.M.; data curation, E.M. and E.V.; writing—original draft preparation, E.M.; writing—review and editing, E.V., T.K. and V.K.; visualization, G.A.P.; supervision, G.A.P. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Brandt, J.; Guo, P.J.; Lewenstein, J.; Dontcheva, M.; Klemmer, S.R. Two Studies of Opportunistic Programming. In Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, New York, NY, USA, 4 April 2009; ACM, pp. 1589–1598.
2. Robillard, M.P.; Maalej, W.; Walker, R.J.; Zimmermann, T. *Recommendation Systems in Software Engineering*; Robillard, M.P., Maalej, W., Walker, R.J., Zimmermann, T., Eds.; Springer: Berlin/Heidelberg, Germany, 2014; ISBN 978-3-642-45134-8.
3. Robillard, M.; Walker, R.; Zimmermann, T. Recommendation Systems for Software Engineering. *IEEE Softw.* **2010**, *27*, 80–86. [CrossRef]
4. Mikić, V.; Ilić, M.; Kopanja, L.; Vesin, B. Personalisation Methods in E-learning-A Literature Review. *Comput. Appl. Eng. Educ.* **2022**, *30*, 1931–1958. [CrossRef]
5. Sivapalan, S.; Sadeghian, A.; Rahnama, H.; Madni, A.M. Recommender Systems in E-Commerce. In Proceedings of the 2014 World Automation Congress (WAC), Waikoloa, HI, USA, 3–7 August 2014; IEEE: Piscataway, NJ, USA, 2014; pp. 179–184.
6. Pakdeetrakulwong, U.; Wongthongtham, P.; Siricharoen, W.V. Recommendation Systems for Software Engineering: A Survey from Software Development Life Cycle Phase Perspective. In Proceedings of the 9th International Conference for Internet

Technology and Secured Transactions (ICITST-2014), London, UK, 8–10 December 2014; IEEE: Piscataway, NJ, USA, 2014; pp. 137–142.
7. Durrani, U.K.; Akpinar, M.; Fatih Adak, M.; Talha Kabakus, A.; Maruf Öztürk, M.; Saleh, M. A Decade of Progress: A Systematic Literature Review on the Integration of AI in Software Engineering Phases and Activities (2013–2023). *IEEE Access* **2024**, *12*, 171185–171204. [CrossRef]
8. Wan, Y.; Bi, Z.; He, Y.; Zhang, J.; Zhang, H.; Sui, Y.; Xu, G.; Jin, H.; Yu, P. Deep Learning for Code Intelligence: Survey, Benchmark and Toolkit. *ACM Comput. Surv.* **2024**, *56*, 1–41. [CrossRef]
9. Jin, H.; Zhou, Y.; Hussain, Y. Enhancing Code Completion with Implicit Feedback. In Proceedings of the 2023 IEEE 23rd International Conference on Software Quality, Reliability and, Security (QRS), Chiang Mai, Thailand, 22 October 2023; IEEE: Piscataway, NJ, USA, 2023; pp. 218–227.
10. Di Grazia, L.; Pradel, M. Code Search: A Survey of Techniques for Finding Code. *ACM Comput. Surv.* **2023**, *55*, 1–31. [CrossRef]
11. Tao, C.; Lin, K.; Huang, Z.; Sun, X. CRAM: Code Recommendation With Programming Context Based on Self-Attention Mechanism. *IEEE Trans. Reliab.* **2023**, *72*, 302–316. [CrossRef]
12. Silavong, F.; Moran, S.; Georgiadis, A.; Saphal, R.; Otter, R. Senatus: A Fast and Accurate Code-to-Code Recommendation Engine. In Proceedings of the 19th International Conference on Mining Software Repositories, Pittsburgh, PA, USA, 23 May 2022; ACM: New York, NY, USA, 2022; pp. 511–523.
13. Hammad, M.; Babur, Ö.; Abdul Basit, H.; Brand, M. van den Clone-Advisor: Recommending Code Tokens and Clone Methods with Deep Learning and Information Retrieval. *PeerJ Comput. Sci.* **2021**, *7*, e737. [CrossRef]
14. Sun, H.; Xu, Z.; Li, X. Code Recommendation Based on Deep Learning. In Proceedings of the 2023 12th International Conference of Information and Communication Technology (ICTech), Wuhan, China, 14–16 April 2023; IEEE: Piscataway, NJ, USA, 2023; pp. 156–160.
15. Wen, W.; Zhao, T.; Wang, S.; Chu, J.; Kumar Jain, D. Code Recommendation Based on Joint Embedded Attention Network. *Soft Comput.* **2022**, *26*, 8635–8645. [CrossRef]
16. Islam, M.M.; Iqbal, R. SoCeR: A New Source Code Recommendation Technique for Code Reuse. In Proceedings of the 2020 IEEE 44th Annual Computers, Software, and Applications Conference (COMPSAC), Madrid, Spain, 13–17 July 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 1552–1557.
17. Abid, S.; Abdul Basit, H.; Shamail, S. Context-Aware Code Recommendation in Intellij IDEA. In Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Singapore, 14–16 November 2022; ACM: New York, NY, USA, 2022; pp. 1647–1651.
18. Jansen, S.; Brinkkemper, S.; Hunink, I.; Demir, C. Pragmatic and Opportunistic Reuse in Innovative Start-up Companies. *IEEE Softw.* **2008**, *25*, 42–49. [CrossRef]
19. Abid, S.; Shamail, S.; Basit, H.A.; Nadi, S. FACER: An API Usage-Based Code-Example Recommender for Opportunistic Reuse. *Empir. Softw. Eng.* **2021**, *26*, 110. [CrossRef]
20. Yu, Y.; Huang, Z.; Shen, G.; Li, W.; Shao, Y. ASTSDL: Predicting the Functionality of Incomplete Programming Code via an AST-Sequence-Based Deep Learning Model. *Sci. China Inf. Sci.* **2024**, *67*, 112105. [CrossRef]
21. Nguyen, T.T.; Nguyen, T.T. PERSONA: A Personalized Model for Code Recommendation. *PLoS ONE* **2021**, *16*, e0259834. [CrossRef]
22. Asaduzzaman, M.; Roy, C.K.; Monir, S.; Schneider, K.A. Exploring API Method Parameter Recommendations. In Proceedings of the 2015 IEEE 31st International Conference on Software Maintenance and Evolution (ICSME 2015), Bremen, Germany, 29 September–1 October 2015; pp. 271–280.
23. Siddiq, M.L.; Casey, B.; Santos, J.C.S. Franc: A Lightweight Framework for High-Quality Code Generation. In Proceedings of the 2024 IEEE International Conference on Source Code Analysis and Manipulation (SCAM), Flagstaff, AZ, USA, 7–8 October 2024; pp. 106–117.
24. Hussain, Y.; Huang, Z.; Zhou, Y.; Wang, S. Boosting Source Code Suggestion with Self-Supervised Transformer Gated Highway. *J. Syst. Softw.* **2023**, *196*, 111553. [CrossRef]
25. Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; de Pinto, H.P.O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. Evaluating Large Language Models Trained on Code. *arXiv* **2021**, arXiv:2107.03374.
26. Rozière, B.; Gehring, J.; Gloeckle, F.; Sootla, S.; Gat, I.; Tan, X.E.; Adi, Y.; Liu, J.; Sauvestre, R.; Remez, T.; et al. Code Llama: Open Foundation Models for Code. *arXiv* **2023**, arXiv:2308.12950.
27. Zan, D.; Chen, B.; Zhang, F.; Lu, D.; Wu, B.; Guan, B.; Yongji, W.; Lou, J.-G. Large Language Models Meet NL2Code: A Survey. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers); Association for Computational Linguistics: Stroudsburg, PA, USA, 2023; pp. 7443–7464.
28. Wu, D.; Ahmad, W.U.; Zhang, D.; Ramanathan, M.K.; Ma, X. Repoformer: Selective Retrieval for Repository-Level Code Completion. *arXiv* **2024**, arXiv:2403.10059.

29. Liu, W.; Yu, A.; Zan, D.; Shen, B.; Zhang, W.; Zhao, H.; Jin, Z.; Wang, Q. GraphCoder: Enhancing Repository-Level Code Completion via Coarse-to-Fine Retrieval Based on Code Context Graph. In Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, Sacramento, CA, USA, 27 October–1 November 2024; ACM: New York, NY, USA, 2024; pp. 570–581.
30. Xia, Y.; Liang, T.; Min, W.; Kuang, L. Improving AST-Level Code Completion with Graph Retrieval and Multi-Field Attention. In Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension, Lisbon, Portugal, 15–16 April 2024; ACM: New York, NY, USA, 2024; pp. 125–136.
31. Liang, M.; Xie, X.; Zhang, G.; Zheng, X.; Di, P.; Jiang, W.; Chen, H.; Wang, C.; Fan, G. RepoGenix: Dual Context-Aided Repository-Level Code Completion with Language Models. In Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, Sacramento, CA, USA, 27 October–1 November 2024; ACM: New York, NY, USA, 2024; pp. 2466–2467.
32. Ding, Y.; Wang, Z.; Ahmad, W.U.; Ramanathan, M.K.; Nallapati, R.; Bhatia, P.; Roth, D.; Xiang, B. CoCoMIC: Code Completion by Jointly Modeling In-File and Cross-File Context. *arXiv* **2022**, arXiv:2212.10007.
33. Bibaev, V.; Kalina, A.; Lomshakov, V.; Golubev, Y.; Bezzubov, A.; Povarov, N.; Bryksin, T. All You Need Is Logs: Improving Code Completion by Learning from Anonymous IDE Usage Logs. In Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Singapore, 14–16 November 2022; ACM: New York, NY, USA, 2022; pp. 1269–1279.
34. Böhme, M.; Soremekun, E.O.; Chattopadhyay, S.; Ugherughe, E.; Zeller, A. Where Is the Bug and How Is It Fixed? An Experiment with Practitioners. In Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering, Paderborn, Germany, 4–8 September 2017; ACM: New York, NY, USA, 2017; pp. 117–128.
35. Winter, E.; Bowes, D.; Counsell, S.; Hall, T.; Haraldsson, S.; Nowack, V.; Woodward, J. How Do Developers Really Feel About Bug Fixing? Directions for Automatic Program Repair. *IEEE Trans. Softw. Eng.* **2023**, *49*, 1823–1841. [CrossRef]
36. Zhang, J.; Wang, C.; Li, A.; Wang, W.; Li, T.; Liu, Y. VulAdvisor: Natural Language Suggestion Generation for Software Vulnerability Repair. In Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, Sacramento, CA, USA, 27 October–1 November 2024; ACM: New York, NY, USA, 2024; pp. 1932–1944.
37. Mahajan, S.; Abolhassani, N.; Prasad, M.R. Recommending Stack Overflow Posts for Fixing Runtime Exceptions Using Failure Scenario Matching. In Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual, 8–13 November 2020; ACM: New York, NY, USA, 2020; pp. 1052–1064.
38. Nguyen, T.; Vu, P.; Nguyen, T. Code Recommendation for Exception Handling. In Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual, 8–13 November 2020; ACM: New York, NY, USA, 2020; pp. 1027–1038.
39. Lee, S.; Lee, J.; Kang, S.; Ahn, J.; Cho, H. Code Edit Recommendation Using a Recurrent Neural Network. *Appl. Sci.* **2021**, *11*, 9286. [CrossRef]
40. Dong, C.; Jiang, Y.; Niu, N.; Zhang, Y.; Liu, H. Context-Aware Name Recommendation for Field Renaming. In Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, Lisbon, Portugal, 14–20 April 2024; ACM: New York, NY, USA, 2024; pp. 1–13.
41. Chakraborty, S.; Ding, Y.; Allamanis, M.; Ray, B. CODIT: Code Editing With Tree-Based Neural Models. *IEEE Trans. Softw. Eng.* **2022**, *48*, 1385–1399. [CrossRef]
42. Nyamawe, A.S.; Liu, H.; Niu, N.; Umer, Q.; Niu, Z. Feature Requests-Based Recommendation of Software Refactorings. *Empir. Softw. Eng.* **2020**, *25*, 4315–4347. [CrossRef]
43. Yu, H.; Jia, X.; Mine, T.; Zhao, J. Type Conversion Sequence Recommendation Based on Semantic Web Technology. In Proceedings of the 2018 IEEE SmartWorld, Ubiquitous Intelligence & Computing, Advanced & Trusted Computing, Scalable Computing & Communications, Cloud & Big Data Computing, Internet of People and Smart City Innovation (SmartWorld/SCALCOM/UIC/ATC/CBDCom/IOP/SCI), Guangzhou, China, 7–11 October 2018; IEEE: Piscataway, NJ, USA, October, 2018; pp. 240–245.
44. Zhou, Y.; Yang, X.; Chen, T.; Huang, Z.; Ma, X.; Gall, H. Boosting API Recommendation With Implicit Feedback. *IEEE Trans. Softw. Eng.* **2022**, *48*, 2157–2172. [CrossRef]
45. Gao, S.; Liu, L.; Liu, Y.; Liu, H.; Wang, Y. API Recommendation for the Development of Android App Features Based on the Knowledge Mined from App Stores. *Sci. Comput. Program.* **2021**, *202*, 102556. [CrossRef]
46. Zhou, Y.; Chen, C.; Wang, Y.; Han, T.; Chen, T. Context-Aware API Recommendation Using Tensor Factorization. *Sci. China Inf. Sci.* **2023**, *66*, 122101. [CrossRef]
47. Li, X.; Liu, L.; Liu, Y.; Liu, H. A Lightweight API Recommendation Method for App Development Based on Multi-Objective Evolutionary Algorithm. *Sci. Comput. Program.* **2023**, *226*, 102927. [CrossRef]
48. Wang, Y.; Chen, L.; Gao, C.; Fang, Y.; Li, Y. Prompt Enhance API Recommendation: Visualize the User's Real Intention behind This Query. *Autom. Softw. Eng.* **2024**, *31*, 27. [CrossRef]

49. Wei, M.; Harzevili, N.S.; Huang, Y.; Wang, J.; Wang, S. Clear: Contrastive Learning for Api Recommendation. In Proceedings of the 44th International Conference on Software Engineering, Pittsburgh, PA, USA, 25–27 May 2022; ACM: New York, NY, USA, 2022; pp. 376–387.
50. Gao, S.; Zhang, L.; Liu, H.; Wang, Y. Which Animation API Should I Use Next? A Multimodal Real-Time Animation API Recommendation Model for Android Apps. *IEEE Trans. Softw. Eng.* **2024**, *50*, 106–122. [CrossRef]
51. Wang, Y.; Liu, H.; Gao, S.; Tang, X. Animation2API: API Recommendation for the Implementation of Android UI Animations. *IEEE Trans. Softw. Eng.* **2023**, *49*, 4411–4428. [CrossRef]
52. Cai, J.; Cai, Q.; Li, B.; Zhang, J.; Sun, X. Application Programming Interface Recommendation for Smart Contract Using Deep Learning from Augmented Code Representation. *J. Softw. Evol. Process* **2024**, *36*, e2658. [CrossRef]
53. Gao, H.; Qin, X.; Barroso, R.J.D.; Hussain, W.; Xu, Y.; Yin, Y. Collaborative Learning-Based Industrial IoT API Recommendation for Software-Defined Devices: The Implicit Knowledge Discovery Perspective. *IEEE Trans. Emerg. Top. Comput. Intell.* **2022**, *6*, 66–76. [CrossRef]
54. Wang, Y.; Zhou, Y.; Chen, T.; Zhang, J.; Yang, W.; Huang, Z. Hybrid Collaborative Filtering-Based API Recommendation. In Proceedings of the 2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS), Hainan, China, 6–10 December 2022; pp. 906–914.
55. Li, Z.; Li, C.; Tang, Z.; Huang, W.; Ge, J.; Luo, B.; Ng, V.; Wang, T.; Hu, Y.; Zhang, X. PTM-APIRec: Leveraging Pre-Trained Models of Source Code in API Recommendation. *ACM Trans. Softw. Eng. Methodol.* **2024**, *33*, 1–30. [CrossRef]
56. Li, K.; Tang, X.; Li, F.; Zhou, H.; Ye, C.; Zhang, W. PyBartRec: Python API Recommendation with Semantic Information. In Proceedings of the 14th Asia-Pacific Symposium on Internetware, Hangzhou, China, 4–6 August 2023; ACM: New York, NY, USA, 2023; pp. 33–43.
57. Chen, Z.; Zhang, T.; Peng, X. A Novel API Recommendation Approach By Using Graph Attention Network. In Proceedings of the 2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS), Hainan, China, 6–10 December 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 726–737.
58. Svyatkovskiy, A.; Zhao, Y.; Fu, S.; Sundaresan, N. Pythia: AI-Assisted Code Completion System. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, Anchorage, AK, USA, 4–8 August 2019; ACM: New York, NY, USA, 2019; pp. 2727–2735.
59. D'Souza, A.R.; Yang, D.; Lopes, C.V. Collective Intelligence for Smarter API Recommendations in Python. In Proceedings of the 2016 IEEE 16th International Working Conference on Source Code Analysis and Manipulation (SCAM), Raleigh, NC, USA, 2–3 October 2016; IEEE: Piscataway, NJ, USA, 2016; pp. 51–60.
60. Nguyen, P.T.; Di Rocco, J.; Di Sipio, C.; Di Ruscio, D.; Di Penta, M. Recommending API Function Calls and Code Snippets to Support Software Development. *IEEE Trans. Softw. Eng.* **2022**, *48*, 2417–2438. [CrossRef]
61. Xie, R.; Kong, X.; Wang, L.; Zhou, Y.; Li, B. HiRec: API Recommendation Using Hierarchical Context. In Proceedings of the 2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE), Berlin, Germany, 28–31 October 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 369–379.
62. Zhao, Y.; Li, L.; Wang, H.; He, Q.; Grundy, J. APIMatchmaker: Matching the Right APIs for Supporting the Development of Android Apps. *IEEE Trans. Softw. Eng.* **2023**, *49*, 113–130. [CrossRef]
63. Thung, F.; Oentaryo, R.J.; Lo, D.; Tian, Y. WebAPIRec: Recommending Web APIs to Software Projects via Personalized Ranking. *IEEE Trans. Emerg. Top. Comput. Intell.* **2017**, *1*, 145–156. [CrossRef]
64. Chen, Y.; Gao, C.; Ren, X.; Peng, Y.; Xia, X.; Lyu, M.R. API Usage Recommendation Via Multi-View Heterogeneous Graph Representation Learning. *IEEE Trans. Softw. Eng.* **2023**, *49*, 3289–3304. [CrossRef]
65. Nguyen, S.; Manh, C.T.; Tran, K.T.; Nguyen, T.M.; Nguyen, T.-T.; Ngo, K.-T.; Vo, H.D. ARist: An Effective API Argument Recommendation Approach. *J. Syst. Softw.* **2023**, *204*, 111786. [CrossRef]
66. Nguyen, P.T.; Di Rocco, J.; Di Ruscio, D.; Ochoa, L.; Degueule, T.; Di Penta, M. FOCUS: A Recommender System for Mining API Function Calls and Usage Patterns. In Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), Montreal, QC, Canada, 25–31 May 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 1050–1060.
67. Zhang, Y.; Chen, X. Explainable Recommendation: A Survey and New Perspectives. *Found. Trends® Inf. Retr.* **2020**, *14*, 1–101. [CrossRef]
68. Ciniselli, M.; Pascarella, L.; Aghajani, E.; Scalabrino, S.; Oliveto, R.; Bavota, G. Source Code Recommender Systems: The Practitioners' Perspective. In Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), Melbourne, Australia, 14–20 May 2023; IEEE: Piscataway, NJ, USA, 2023; pp. 2161–2172.
69. McMillan, C.; Grechanik, M.; Poshyvanyk, D.; Fu, C.; Xie, Q. Exemplar: A Source Code Search Engine for Finding Highly Relevant Applications. *IEEE Trans. Softw. Eng.* **2012**, *38*, 1069–1087. [CrossRef]
70. Ribeiro, M.T.; Singh, S.; Guestrin, C. "Why Should I Trust You"? In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, 13–17 August 2016; ACM: New York, NY, USA, 2016; pp. 1135–1144.

71. Lundberg, S.; Lee, S.-I. A Unified Approach to Interpreting Model Predictions. *Adv. Neural Inf. Process. Syst.* **2017**, *30*, 4768–4777.
72. Ying, R.; Bourgeois, D.; You, J.; Zitnik, M.; Leskovec, J. GNNExplainer: Generating Explanations for Graph Neural Networks. In Proceedings of the Advances in Neural Information Processing Systems, Vancouver, BC, Canada, 8–14 December 2019; Volume 32.
73. Huang, Q.; Yamada, M.; Tian, Y.; Singh, D.; Chang, Y. GraphLIME: Local Interpretable Model Explanations for Graph Neural Networks. *IEEE Trans. Knowl. Data Eng.* **2023**, *35*, 6968–6972. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

Reproduced with permission of copyright owner. Further reproduction prohibited without permission.